

從零打造一個 MySQL-based RAG 系統

VECTOR 型別實戰、工程取捨與 pgvector 對照

Hank (綠豆湯 / litotom)

COSCUP 2026 • MySQL Track



投影片與範例程式碼 • <https://github.com/hanktom/coscup-mysql>

關於我

- 軟體開發 30 年：程式設計師 → 軟體部經理 → 技術顧問 → 總監
- 「綠豆湯」技術部落格 (litotom) 發起人
- 全球第 18 位 Google Certified Android Developer
- 今天的身分：幫 **DBA** 說話的人

綠豆選物的客服危機

虛構電商「綠豆選物 LitoShop」，客服每天被問：

「退款多久會到？」

「錢什麼時候退回我卡裡？」

「刷退了怎麼帳單上還沒看到？」

同一個答案，一百種問法。

老闆說：上 AI 客服。

於是有了那場會議

架構師：「要做 RAG，我們得加一座向量資料庫。Pinecone？Qdrant？還是上 pgvector？」

DBA：「……我們不是有 MySQL 嗎？」

（會議室安靜了三秒）

這場演講，就是那三秒之後的完整技術答辯。

今天要回答的問題

「公司只有 MySQL，能不能不搬資料庫，把 RAG 做起來？」

- MySQL 9.x VECTOR 型別到底能做什麼、不能做什麼
- 從零跑起一個 AI 客服（全開源、可重現）
- 同資料同查詢，與 pgvector 誠實對照
- 一個帶得走的選型決策框架

不談：LLM 訓練、向量數學、HeatWave 等雲端專屬功能

RAG，一頁講完



- Embedding：把語意變成向量（今天用 **bge-m3**，1024 維，MIT 授權）
- 檢索：在一堆向量裡找最近的 K 個 —— 這就是資料庫的工作
- 今天的主角只有一個：檢索層的資料庫

RAG 選型討論的現況

大家想到的儲存層：

Pinecone | Qdrant | Milvus | Weaviate | **PostgreSQL + pgvector**

沒有人提 MySQL。

但現實是——

MySQL 是企業既有 stack 裡最常見的 OLTP 資料庫。

你的訂單、會員、商品，本來就住在裡面。

「就用 MySQL」的真實誘因

- 不搬資料：知識庫和交易資料同一座庫，直接 JOIN
- 不加維運面：沒有新資料庫要備份、監控、升級、待命
- 權限與稽核：既有的帳號體系、audit 流程直接沿用
- 團隊技能：DBA 已經在了

這些不是懶惰，是**工程成本的真實計算**。

問題只剩一個：MySQL 做得到嗎？

MySQL 9.x 的 VECTOR 型別

MySQL 9.0 (2024-07) 正式引入：

```
CREATE TABLE faq_chunks (  
  id          INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,  
  category    VARCHAR(32) NOT NULL,  
  content     TEXT        NOT NULL,  
  embedding   VECTOR(1024) NOT NULL      -- ◀  
);
```

- 每維 4 bytes (float32) 的 binary 儲存，上限 16,383 維
- 1024 維 $\approx$ 4KB/列，10 萬筆 $\approx$ 400MB
- InnoDB、binlog、replication 一切照舊——它就是個欄位

社群版給你的函式：就這三個

```
SELECT STRING_TO_VECTOR('[0.1, 0.2, 0.3]');  --  
SELECT VECTOR_TO_STRING(embedding);          --  
SELECT VECTOR_DIM(embedding);                --
```

那……計算相似度的函式呢？

Live：算個距離吧

```
mysql> SELECT DISTANCE(  
->   embedding,  
->   STRING_TO_VECTOR('[0.1, 0.2, ...]'),  
->   'COSINE'  
-> ) FROM faq_chunks LIMIT 1;
```

```
ERROR 1305 (42000): FUNCTION ragdemo.DISTANCE does not exist
```

官方文件：「*DISTANCE()* ... available only for MySQL HeatWave on OCI」

現況總結（至 MySQL 9.7，2026-04 GA）

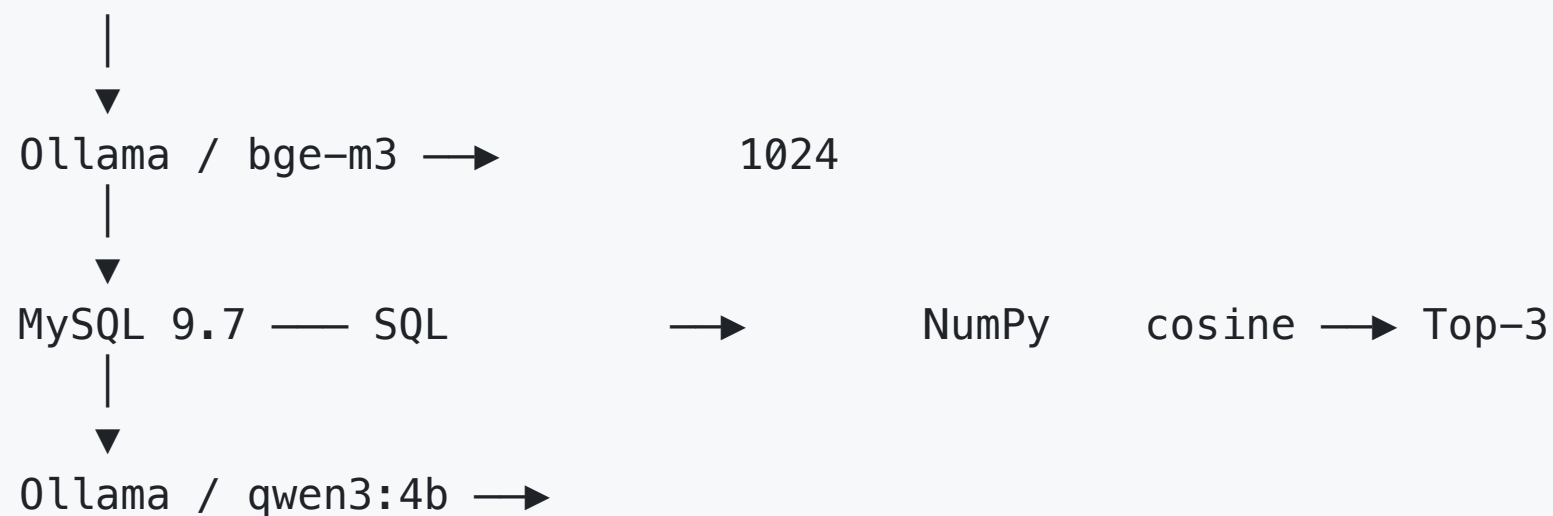
能力	社群版	HeatWave (OCI)
VECTOR 型別儲存	有	有
轉換函式（STRING_TO_VECTOR 等）	有	有
DISTANCE() 距離函式	無	有
ANN 索引（HNSW / IVF）	無	有

型別給了，引擎沒給。

「像一台有座椅和方向盤、但沒有引擎的車。」

那就自己裝引擎——距離計算，搬到應用端。

Demo 架構：綠豆選物 AI 客服



全開源、全本地、`docker compose up` 可重現

知識庫：合成的「綠豆選物」客服 FAQ（CC0，43 chunks / 10 類別）

檢索策略：MySQL 當儲存層，Python 當引擎

```
cur.execute("SELECT id, title, content, embedding FROM faq_chunks")
rows = cur.fetchall()

# VECTOR          float32 binary →      frombuffer
mat = np.frombuffer(b"".join(r[3] for r in rows), dtype=np.float32)
mat = mat.reshape(len(rows), 1024)

scores = mat @ query_vec          #      normalize      = cosine
topk    = np.argsort(-scores)[:3]
```

兩個關鍵細節：

- 取 **原始 binary**，不要 `VECTOR_TO_STRING`（省一次文字解析）
- embedding 先 `normalize`，`cosine` 退化成一個矩陣乘法

▶ Live Demo I

綠豆選物 AI 客服 on MySQL

「退款多久會到？」 → 「錢什麼時候退回我卡裡？」
換尺寸要收費嗎？ | ATM 解除分期是詐騙嗎？

Demo I 小結

- 43 個知識 chunk、幾百條 FAQ 規模：檢索 2-3 ms
- 完全不同的問法，語意檢索照樣命中——關鍵字搜尋做不到
- RAG 讓 LLM 「說對的話」：答案有來源、可稽核

在這個規模，MySQL 就是夠用。

會議可以散會了嗎？

還不行：知識庫會長大

客服 FAQ 只是開始。接下來會進來的是：

- 全站商品說明（數萬筆）
- 歷史客服工單（數十萬筆）
- 站內文章、規格表、評價……

10 萬筆向量時，剛剛那套還撐得住嗎？

Benchmark：同資料、同查詢

(7GB RAM 容器實測，p50；現場數據以講者 VM 重跑為準)

情境	1 萬筆	10 萬筆
MySQL 全掃描 + 應用端排序	242 ms	2,144 ms
MySQL category 過濾 (砍到 1/10)	38 ms	244 ms
pgvector 無索引 (exact)	39 ms	469 ms
pgvector HNSW	2.2 ms	9.7 ms

2 秒是怎麼來的？

每一次查詢，MySQL 方案都在做這件事：

把 400MB 的向量搬出資料庫，再全部算一遍。

- 瓶頸不在 NumPy（矩陣乘法只佔 ~5%）
- 瓶頸在 網路傳輸 + 驅動反序列化
- 資料量 $\times 10$ ，延遲就 $\times 10$ ——線性，沒有懸念

pgvector 為什麼快？距離計算留在資料庫裡，HNSW 讓它不必全算。

MySQL 陣營的規避術

- Metadata 粗過濾（最有效）：客服天然有 `category` / `tenant` 維度
→ 一個 `WHERE` 砍掉 90% 候選，2,144ms → 244ms
- Partition by tenant：多租戶 SaaS 場景，每租戶各自小全掃
- 覆蓋索引：`(category, id)` + 二段式取向量，減少搬運
- Read replica：向量查詢分流，別跟交易搶 buffer pool

誠實說：這些都是「延後失守」，不是 ANN 的替代品。

對照組登場：PostgreSQL + pgvector

- 開源 extension (PostgreSQL License)，目前 0.8.x
- `vector` 型別 + 6 種距離運算子 + HNSW / IVFFlat 索引
- 0.8 的 iterative index scan：過濾條件與 ANN 索引的配合更聰明

```
SELECT doc_id, title, embedding <=> :query_vec AS dist
FROM   faq_chunks
ORDER BY embedding <=> :query_vec           --          DB
LIMIT 5;                                   --   HNSW   ANN
```

同一支 chatbot，`--db pg` 一個參數切換。

▶ Live Demo II

同一份資料，搬進 pgvector

ORDER BY embedding <=> q | EXPLAIN ANALYZE

HNSW 索引建立前 vs 後

EXPLAIN 前後（10 萬筆）

建索引前：

```
Limit ... Sort ... Seq Scan on faq_chunks  
Execution Time: ~470 ms
```

CREATE INDEX ... USING hnsw (embedding vector_cosine_ops) 之後：

```
Limit ... Index Scan using idx_faq_embedding on faq_chunks  
Execution Time: ~10 ms
```

一條 DDL，差 47 倍。這就是 ANN 索引的價值。

一頁對照表

	MySQL 9.7 社群版	PostgreSQL + pgvector 0.8
向量型別	VECTOR (≤16,383 維)	vector / halfvec / sparsevec
距離計算	無 (應用端自理)	6 種運算子，DB 內完成
ANN 索引	無	HNSW / IVFFlat
過濾 + 向量檢索	SQL 過濾 → app 排序	iterative index scan
額外維運成本	零 (既有 stack)	多一座 PG (若原本沒有)
交易資料 JOIN	同庫直接 JOIN	要看你資料住哪

pgvector 也不是免費午餐

- **HNSW build**：10 萬筆 1024 維約需分鐘級 + 足夠的 `maintenance_work_mem`（我們在 demo 環境就先踩了 shm 不足的坑）
- **ANN = 近似**：recall 不是 100%，`ef_search` 要調
- **多一座資料庫**：備份、HA、升級、監控、人力——當初不想搬的理由都還在
- 資料要**同步**過去：CDC 或雙寫，一致性是你的責任

選 pgvector 不是「按下開關」，是「接下一座資料庫」。

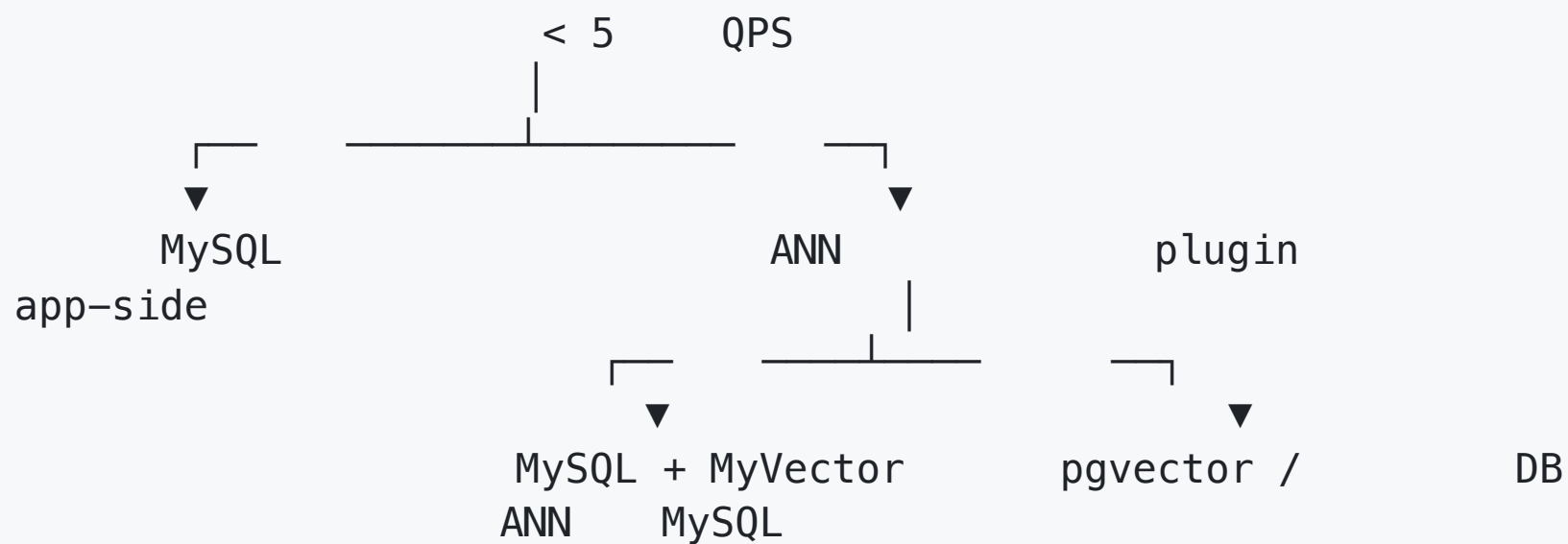
第三條路：社群自己裝引擎 — MyVector

開源 (GPLv2) 的 MySQL plugin/component，HNSW ANN 直接進 MySQL：

```
CREATE TABLE faq (  
  id INT PRIMARY KEY,  
  vec VARBINARY(4096) COMMENT 'MYVECTOR(type=HNSW,dim=1024,dist=COSINE)'  
);  
CALL mysql.myvector_index_build('ragdemo.faq.vec', 'id');  
  
SELECT id, myvector_row_distance() AS d FROM faq  
WHERE MYVECTOR_IS_ANN('ragdemo.faq.vec', 'id', @qvec, 10);
```

- binlog 同步索引、支援 MySQL 8.0 / 8.4 / 9.7
- 定位：想留在 MySQL、又真的需要 ANN 的人
- 風險自評：第三方 plugin 的升級週期、支援量能、社群規模

三岔路决策框架



決策因素，不只有 latency

因素	偏向 MySQL	偏向 pgvector / 向量 DB
向量規模	< 5 萬	> 10 萬且持續成長
查詢延遲要求	秒級可接受 / 離線	線上 p95 < 100ms
過濾維度	天然多租戶 / 分類	全域跨界檢索
團隊現狀	只有 MySQL DBA	本來就有 PG
資料引力	要跟交易資料 JOIN	知識庫獨立

沒有正確答案，只有你的情境。

今天帶走的三件事

1. 一份跑得起來的 demo

`docker compose up` → 綠豆選物 AI 客服 (MySQL + pgvector 雙後端)

2. 一張誠實的對照表

含 benchmark 腳本，回家用自己的資料重跑

3. 一個決策框架

下次會議室安靜三秒時，你有答案

社群共好

- 本 repo：程式碼 **Apache-2.0**、內容 **CC BY-SA 4.0**、資料集 **CC0**
- MySQL 的 vector search 還在 roadmap 上——
去官方管道投票、留言、寫 blog，讓 Oracle 知道社群要什麼
- MariaDB 11.7 的原生 VECTOR 索引、MyVector——生態在動，故事未完

謝謝！

Q&A

repo : `github.com/<your-repo>` (QR code)

綠豆湯 / litotom

程式碼 Apache-2.0 • 投影片 CC BY-SA 4.0 • 資料集 CC0